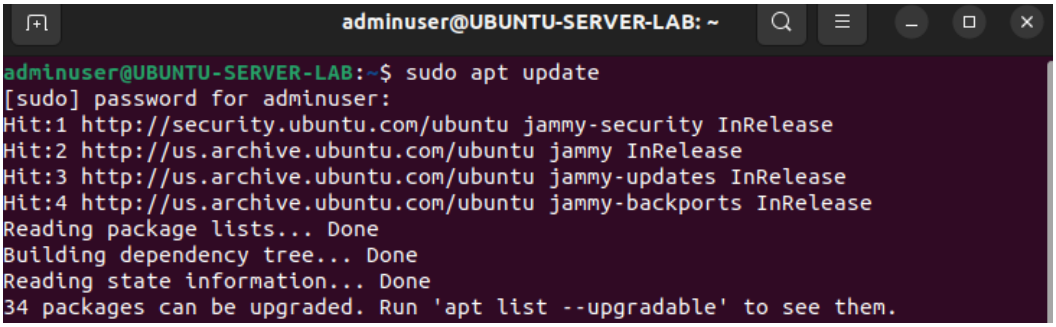
GENERAL SYSTEM TASKS:

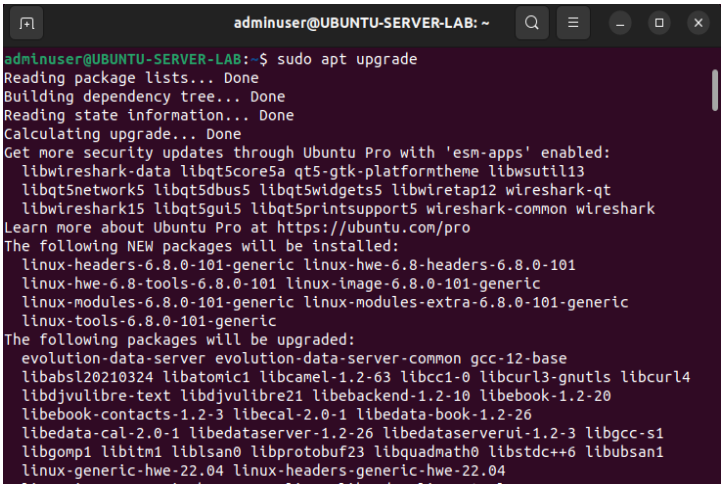
1. Retrieving Updates:

- **Command:** *sudo apt update*
- **Purpose:** Refreshes Ubuntu's package list, so the system knows what updates are available.



2. Upgrade your system:

- **Command:** *sudo apt upgrade*
- **Purpose:** This actually downloads and installs the updates found in the previous step.



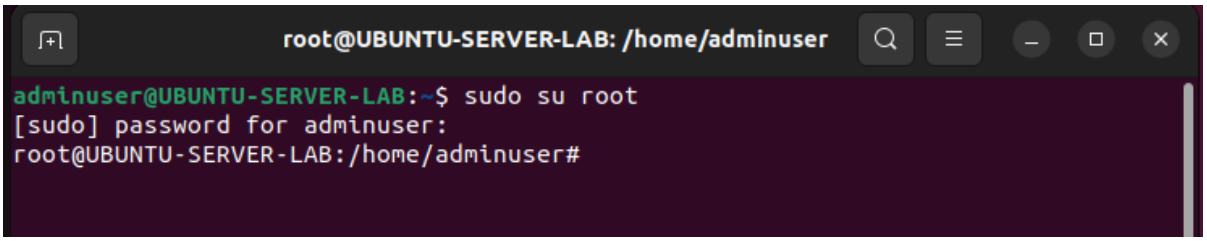
3. Reboot your system:

- **Command:** *sudo reboot*
- **Purpose:** Restarts the VM to finish applying updates.

USER TASKS:

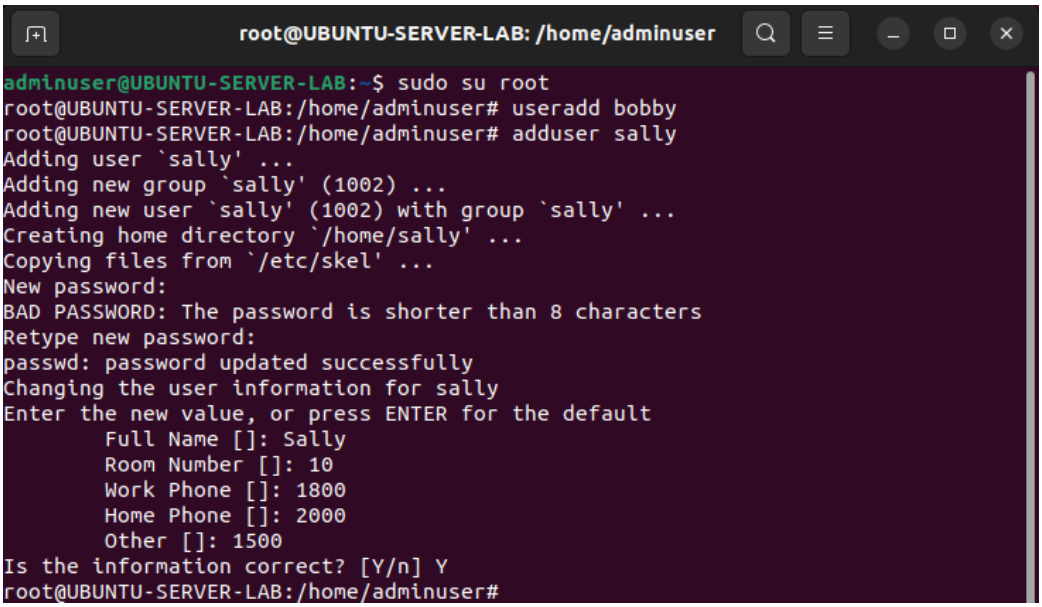
4. Switching to Root

- **Command:** *sudo su root*
- **Purpose:** Change the current user to the system root.
- It also changes the prompt from a **$** (standard user) to a **#** (root user), and the username to root.



5. Creating Users: Bobby(useradd) and sally(adduser)

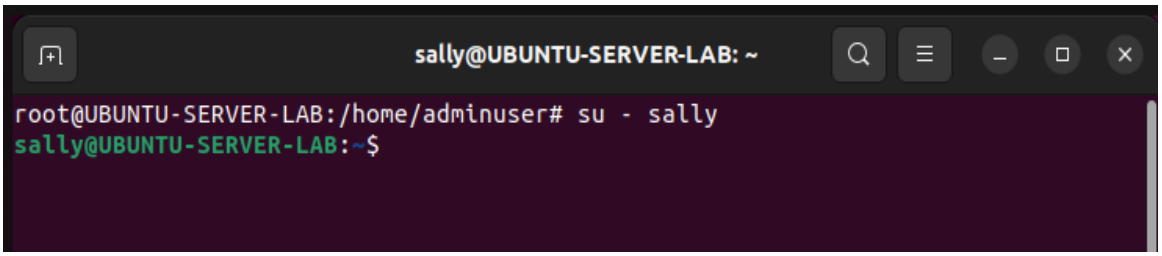
- **Commands:** *useradd bobby* | *adduser sally*
- **Purpose:** To create two users using different command.
- **Difference between useradd and adduser:**
  - **useradd:** It is a low-level utility that creates the user but created the account without setting password and creating home directory.
  - **adduser:** It is a high-level interactive script that is more user friendly. It also creates the home the home directory automatically, asks to set a password, and other optional details such as full name, room number, etc.



6. Switching to Sally

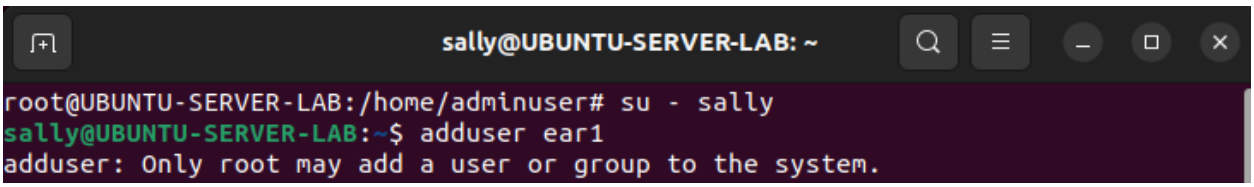
- **Command:** ***su - sally***
- **Purpose:** Change the current user from system root to the new user sally.

- The command changes the prompt from root to sally and went back to **$** since this user is not root by default.

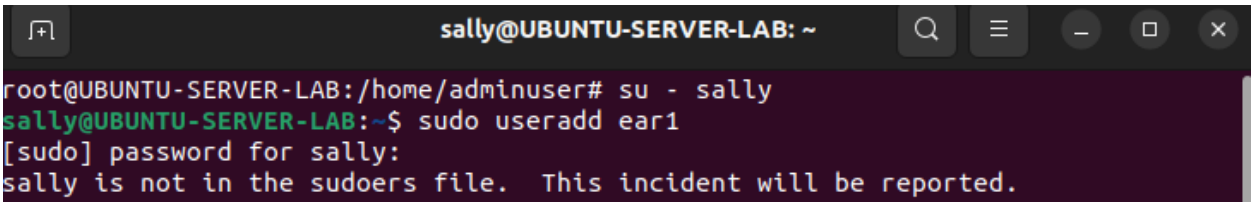


7. While logged in as sally, try creating user ear1

- **Command:** *useradd ear1*
- **Purpose:** To create a new user with sally's account.
- This command fails with an error message "Only root may add a user" since creating users requires root privileges



- To allow sally to create a new user, we will need to give sally sudo privileges, without sudo even if sally tried to create user with **command: *sudo useradd ear1,*** it will return an error saying "not in the sudoers file".

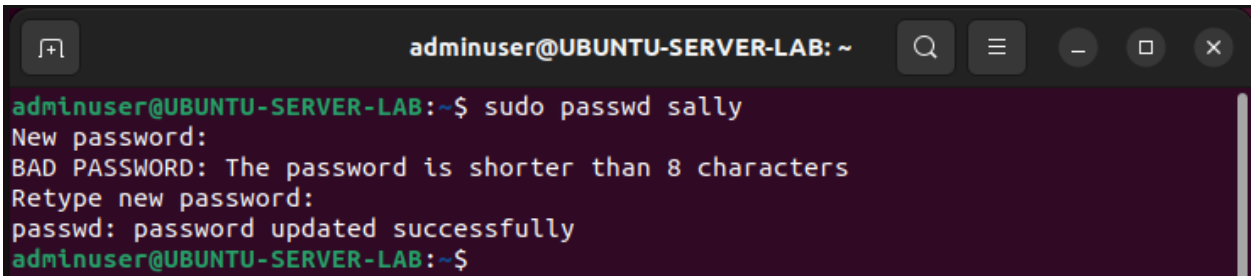


8. Exit back to own account and delete user bobby

- **Command:** *sudo userdel bobby*
- **Purpose:** To remove the user bobby using owner accout.
- *The command will still need sudo to run the *userdel* because the admin or the main account is not the same as a root account but it is the most trusted user that may become a root.

9. Change sally's password

- **Command:** *su passwd sally*
- **Purpose:** To change the password of user sally.
- The command updates sally's password hash in the **/etc/shadow.** It doesn't show the password while typing for security and also returns caution message regarding the password.

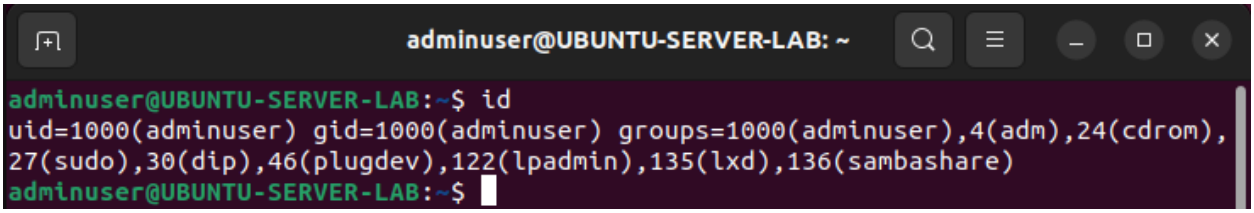


10. Why it's bad practice to stay logged in as root?

- Staying logged in as root is risky because root can do anything on the system with no warnings.
- Even a small mistake like deleting the wrong folder, running a bad script, or mistyping a command can break the whole system or make it insecure.
- It's safer to use a normal account and only use sudo when admin privileges are needed.

11. Change sally's password

- **Command:** *id*
- **Purpose:** To show the current account's identity info like UID (user ID number), GID (primary group ID), and other groups which the user belong to.



<u>GROUP TASKS:</u>

12. What groups does ubuntu belong to

- **Command:** *groups*
- **Purpose:** To display what groups the ubuntu belongs to.

```
adminuser@UBUNTU-SERVER-LAB: ~
adminuser@UBUNTU-SERVER-LAB:~$ groups
adminuser adm cdrom sudo dip plugdev lpadmin lxd sambashare
adminuser@UBUNTU-SERVER-LAB:~$
```

13. Give sally sudo privileges, then create a user while logged in as sally

- **Command:** *sudo usermod -aG sudo sally*
- **Purpose:** To allows the user sally to run admin commands using sudo.
- The command adds sally to the sudo group using the flags **-a (append)** and **-G (add to these groups)**.
- After the command switch to sally using **command: *su – sally*** and then add the user using **command: *sudo useradd ear1*.** Now the command should run since sally is one of the sudoers.

```
sally@UBUNTU-SERVER-LAB: ~
adminuser@UBUNTU-SERVER-LAB:~$ sudo usermod -aG sudo sally
adminuser@UBUNTU-SERVER-LAB:~$ su - sally
Password:
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

sally@UBUNTU-SERVER-LAB:~$ sudo useradd ear1
[sudo] password for sally:
sally@UBUNTU-SERVER-LAB:~$
```

14. Log out of sally, create a group called cybersec

- **Command:** *sudo groupadd cybersec*
- **Purpose:** To create a new group which can manage permissions for multiple users.

```
adminuser@UBUNTU-SERVER-LAB: ~
adminuser@UBUNTU-SERVER-LAB:~$ sudo groupadd cybersec
adminuser@UBUNTU-SERVER-LAB:~$
```

15. Add sally to group cybersec

- **Command:** *sudo usermod -aG cybersec sally*
- **Purpose:** To add sally in the cybersec group.

```
adminuser@UBUNTU-SERVER-LAB: ~
adminuser@UBUNTU-SERVER-LAB:~$ sudo usermod -aG cybersec sally
adminuser@UBUNTU-SERVER-LAB:~$
```

16. Check which groups sally belongs to

- **Command:** *groups sally* | *id sally*
- **Purpose:** To check which groups the user sally belongs to.
- This can be done in multiple ways. Some of the other ways are:
    - Using grep with username in the /etc/group directory (**Command:** ***grep sally /etc/group).***
    - Using grep with username in the /etc/passwd directory (**Command:** ***grep sally /etc/passwd).***

```
adminuser@UBUNTU-SERVER-LAB: ~
adminuser@UBUNTU-SERVER-LAB:~$ groups sally
sally : sally sudo cybersec
adminuser@UBUNTU-SERVER-LAB:~$ id sally
uid=1002(sally) gid=1002(sally) groups=1002(sally),27(sudo),1004(cybersec)
adminuser@UBUNTU-SERVER-LAB:~$
```

PERMISSIONS AND ACCESS CONTROL LISTS:

17. Create lab1 directory and view permissions

- **Command :** *mkdir lab1* | *ls -la lab1*
- **Purpose:** To create a folder for the lab and check its permissions.
- The **mkdir** utility creates the directory and **ls -la** shows the directory details such as permissions, owner, group, date.
- The **ls** command shows permissions for lab1 directory with permissions drwxrwxr-x (owner can read/write/execute, group can read/write/execute, others can read/execute).

```
adminuser@UBUNTU-SERVER-LAB: ~
adminuser@UBUNTU-SERVER-LAB:~$ mkdir lab1
adminuser@UBUNTU-SERVER-LAB:~$ ls -la lab1
total 8
drwxrwxr-x  2 adminuser adminuser 4096 Feb 27 14:09 .
drwxr-x--- 15 adminuser adminuser 4096 Feb 27 14:09 ..
```

18. Inside lab1, create helloWorld bash script + make executable

- **Commands:** *cd lab1* | *nano helloWorld* | *chmod +x helloWorld* | *./helloWorld*
- **Purpose:** To create a basic script and give it executable permission.
- To make the script executable use **command:** ***chmod +x helloWorld***, then run the script using the **command:** ***./helloWorld***
- The **chmod +x** adds execute permission to the file and **./filename** runs the script from the current directory.

```
adminuser@UBUNTU-SERVER-LAB:~$ cd lab1
adminuser@UBUNTU-SERVER-LAB:~/lab1$ nano helloWorld
adminuser@UBUNTU-SERVER-LAB:~/lab1$ sudo chmod +x helloWorld
[sudo] password for adminuser:
adminuser@UBUNTU-SERVER-LAB:~/lab1$ ./helloWorld
Hello World!
adminuser@UBUNTU-SERVER-LAB:~/lab1$
```

- In the nano editor, we put **#!/bin/bash** at the top of a script. That line is called a *shebang*, and it tells Linux which program should run the script. Without it, the system may not know how to interpret the commands, or it may use the wrong shell.

```
adminuser@UBUNTU-SERVER-LAB: ~/lab1
  GNU nano 6.2                    helloWorld *
#!/bin/bash
echo "Hello World!"
```

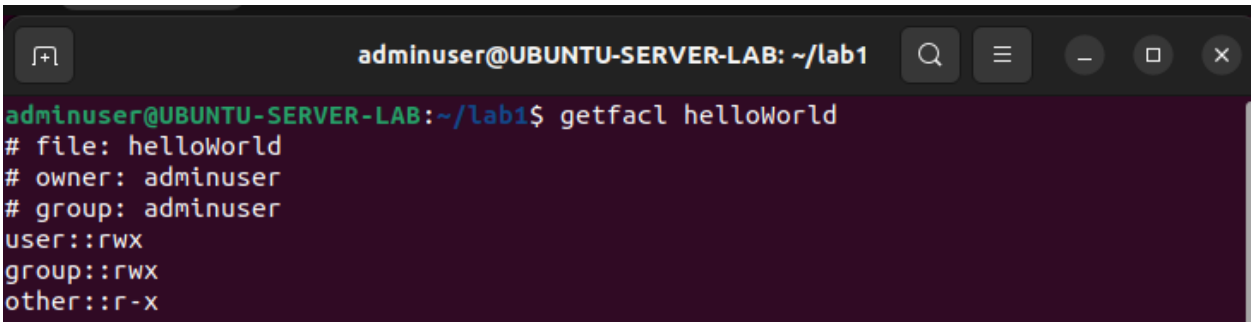
19. Check permissions and add group write + execute

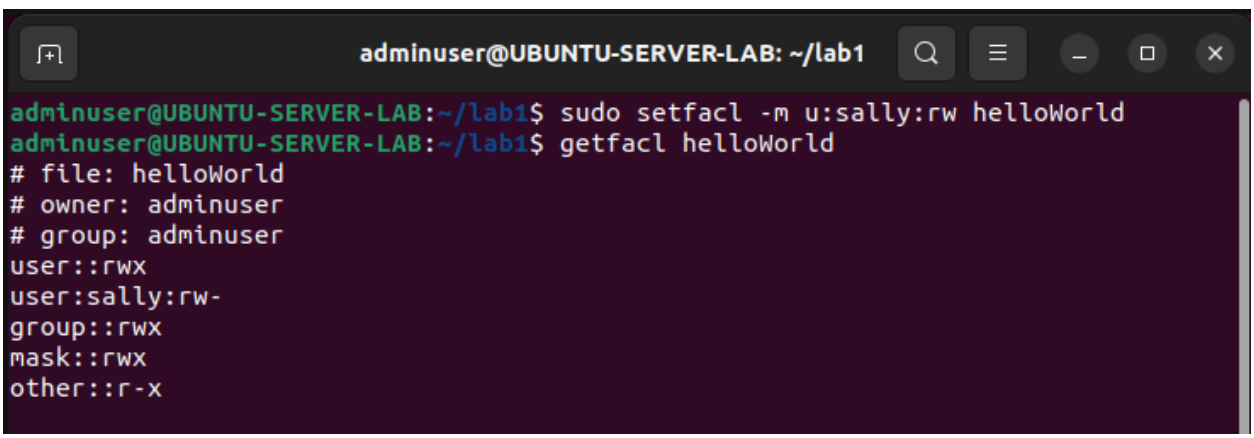
- **Commands:** ***ls -l helloWorld*** | ***chmod g+wx helloWorld*** | ***ls -l helloWorld***
- **Purpose:** To see current permissions of the script helloWorld and then modify them so the group has more access.
- The **chmod g+wx** adds write execute permission of the file to the group.

```
adminuser@UBUNTU-SERVER-LAB: ~/lab1
adminuser@UBUNTU-SERVER-LAB:~/lab1$ ls -l helloWorld
-rwxrwxr-x 1 adminuser adminuser 32 Feb 27 14:11 helloWorld
adminuser@UBUNTU-SERVER-LAB:~/lab1$ sudo chmod g+wx helloWorld
adminuser@UBUNTU-SERVER-LAB:~/lab1$ ls -l helloWorld
-rwxrwxr-x 1 adminuser adminuser 32 Feb 27 14:11 helloWorld
```

20. View ACL with getfacl

- **Command:** *getfacl helloWorld*
- **Purpose:** To view ACL entries, which are extra permissions on a file or directory.
- Shows the file owner, group, and any ACL rules applied to specific users/groups which



21. Create lab1 directory and view permissions

- **Command:** *setfacl -m u:sally:rw helloWorld* | *getfacl helloWorld*
- **Purpose:** To Modify group permissions and set a specific ACL for user sally.
- The **setfacl -m** command modifies the ACL**, u:sally:rw** means user sally gets read/write permissions to the file helloWorld.
- The **getfacl** command then confirms if the ACL is applied or not.